

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

“Программирование”

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Работа с файлами и структуры данных на основе указателей

Вариант: 3-3-9-6

Выполнила:

Студентка гр. 3148 Жук Анастасия

Проверил:

Волков А. Г.

Санкт-Петербург
2024г.

Задание

Разработать на языке C для ОС Linux программу, которая читает из файла заданного формата состояние заданной структуры данных, выполняет манипуляции со структурой в памяти, считывая команды из стандартного потока ввода, и сохраняет результат в файл.

Тип списка: XOR-связный список

Формат данных в строке: MAC-адрес

Дополнительная команда: reorder_dsc (расположить элементы списка по убыванию)

Тип файла:

Содержимое файла:

В файле последовательно располагаются группы из трех полей: индекс строки, размер строки и данные строки. Порядок строк в списке определяется индексами.

Индексы $i_0, \dots, i_{N-1}$ (целые без знака [2 байта]). Индексы соответствующих строк

$s_0, \dots, s_{N-1}$, определяющие порядок строк в списке.

Размеры строк $n_0, \dots, n_{N-1}$ (целые без знака [2 байта]). Размеры строк в байтах.

Строки $s_0, \dots, s_{N-1}$. Строки в кодировке UTF-8 без нулевого байта в конце.

Makefile

```
1 .PHONY: all clean
2
3 APP=lab4avzN3148
4 CFLAGS=-Wall -Wextra -Werror -g
5
6 all: $(APP)
7
8 $(APP):
9     gcc -o $(APP) $(CFLAGS) $(APP).c
10
11 clean:
12     rm $(APP)
```

Примеры работы программы на различных исходных данных

```
user@user-HP-Laptop-15s-eq3xxx:~$ make all
gcc -o lab4avzN3148 -Wall -Wextra -Werror -g lab4avzN3148.c
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 -v
Жук Анастасия Валерьевна, гр. N3148
Вариант: 3-3-9-6
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input
dump

0x57f75070e510 0x57f75070e540 16:16:16:16:16:16
0x57f75070e540 0x60 17:17:17:17:17:17
0x57f75070e570 0x57f75070e540 14:14:14:14:14:14

pop_front
dump

0x57f75070e540 0x57f75070e570 17:17:17:17:17:17
0x57f75070e570 0x57f75070e540 14:14:14:14:14:14

pop_back
dump

0x57f75070e540 (nil) 17:17:17:17:17:17

push_front ab:12:cd:34:ef:56 11:11:11:11:11:11
dump

0x57f75070e510 0x57f75070e570 ab:12:cd:34:ef:56
0x57f75070e570 0x50 11:11:11:11:11:11
0x57f75070e540 0x57f75070e570 17:17:17:17:17:17

push_back 25:96:11:74:35:65 aa:aa:aa:aa:aa:aa
dump

0x57f75070e510 0x57f75070e570 ab:12:cd:34:ef:56
0x57f75070e570 0x50 11:11:11:11:11:11
0x57f75070e540 0xab0 17:17:17:17:17:17
0x57f75070efc0 0xab0 25:96:11:74:35:65
0x57f75070eff0 0x57f75070efc0 aa:aa:aa:aa:aa:aa

reorder_dsc
dump

0x57f75070f060 0x57f75070f090 ab:12:cd:34:ef:56
0x57f75070f090 0xa0 aa:aa:aa:aa:aa:aa
0x57f75070f0c0 0x60 25:96:11:74:35:65
0x57f75070f0f0 0x1e0 17:17:17:17:17:17
0x57f75070f120 0x57f75070f0f0 11:11:11:11:11:11

user@user-HP-Laptop-15s-eq3xxx:~$
```

```
user@user-HP-Laptop-15s-eq3xxx:~$ cat test.txt
push_front 12:12:12:12:12:12 kk:lk:15:12:k5:12 15:15:15:15:15:15 aa:aa:aa:aa:aa:aa
reorder_dsc
dump
pop_front
pop_back
pop_front
pop_back
pop_front
pop_back
pop_front
pop_back
push_back 12:15:85:95:26:36
dump
```

```
user@user-HP-Laptop-15s-eq3xxx:~$ cat test.txt | ./lab4avzN3148 input
```

Ошибка: некорректный тип данных kk:lk:15:12:k5:12.

```
0x58e3ce7d6bd0 0x58e3ce7d6c00 aa:aa:aa:aa:aa:aa
0x58e3ce7d6c00 0x7e0 17:17:17:17:17:17
0x58e3ce7d6c30 0x60 16:16:16:16:16:16
0x58e3ce7d6c60 0xa0 15:15:15:15:15:15
0x58e3ce7d6c90 0xa0 14:14:14:14:14:14
0x58e3ce7d6cc0 0x58e3ce7d6c90 12:12:12:12:12:12
```

```
0x58e3ce7d6c60 (nil) 12:15:85:95:26:36
```

```
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input < test.txt
```

Ошибка: некорректный тип данных kk:lk:15:12:k5:12.

```
0x6330df585bd0 0x6330df585c00 aa:aa:aa:aa:aa:aa
0x6330df585c00 0x7e0 17:17:17:17:17:17
0x6330df585c30 0x60 16:16:16:16:16:16
0x6330df585c60 0xa0 15:15:15:15:15:15
0x6330df585c90 0xa0 14:14:14:14:14:14
0x6330df585cc0 0x6330df585c90 12:12:12:12:12:12
```

```
0x6330df585c60 (nil) 12:15:85:95:26:36
```

```
user@user-HP-Laptop-15s-eq3xxx:~$ █
```

```
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148
```

Ошибка: не задано имя файла.

```
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input
```

magic

Ошибка: неподдерживаемая команда magic.

```
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input < test1.txt
```

Ошибка: неподдерживаемая команда magic.

```
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input
```

pop_front 123

Ошибка: лишние аргументы у команды pop_front.

```
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input
```

pop_back 123

Ошибка: лишние аргументы у команды pop_back.

```
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input
```

```
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input
reorder_dsc 123
Ошибка: лишние аргументы у команды reorder_dsc.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input
dump 123 123
Ошибка: лишние аргументы у команды dump.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148 input 123
Ошибка: много аргументов.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab4avzN3148
Ошибка: не задано имя файла.
```

Исходный текст программы с комментариями

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>
#include <ctype.h>
#include <stdint.h>
#include <errno.h>

#include <time.h>
#include <inttypes.h>
#include <assert.h>

#include <unistd.h>
#include <sys/types.h>

extern int errno;

//виды ошибок
#define NO_ERROR 0 //программа работает корректно
#define ERROR_NO_FILE 1 //не удалось открыть файл
#define ERROR_WRONG_TYPE 2 //неверный тип строк
#define ERROR_DONT_OPEN_FILE 3 //не удалось открыть файл
#define ERROR_UNNECESSARY_ARGUMENTS 4 //лишние аргументы
#define ERROR_BAD_OPERATE 5 //неподдерживаемая операция
#define ERROR_MANY_ARGUMENTS 6 //много аргументов передается при запуске
#define ERROR_WRONG_TYPE_IN_FILE 7 //неверный тип строк в исходном файле

//создание элемента XOR-связного списка
typedef struct list_item_t {
    uintptr_t link;
    char data[17];
} list_item_t;

//создание XOR-связного списка
typedef struct list_t {
    struct list_item_t *head,
    *tail;
} list_t;

//занесение нового элемента списка в конец
void push_back(list_t *lst, char *s) {
    list_item_t *new_item = malloc(sizeof(*new_item)); //новый элемент
    new_item->link = 0;
    strcpy(new_item->data, s);
    if (lst->tail) {
```

```

    lst->tail->link = (uintptr_t)lst->tail->link ^ (uintptr_t)new_item; //сдвигаем текущий элемент
tail на n+1 позицию (теперь элемент связан только с new_item)
    new_item->link = (uintptr_t)lst->tail; //обновляем link new_item, чтобы он был связан с n+1
элементом списка
    lst->tail = new_item; //обновляем адрес tail, чтобы теперь он указывал на новый элемент
}
else { //случай, когда список пустой
    assert(!lst->head);
    lst->head = lst->tail = new_item;
}
}

```

//занесение нового элемента списка в начало

```

void push_front(list_t *lst, char *s) { //lst - список, s - строка, которую нужно внести
    list_item_t *new_item = malloc(sizeof(*new_item)); //новый элемент
    new_item->link = 0;
    strcpy(new_item->data, s);
    if (lst->head) {
        lst->head->link = (uintptr_t)lst->head->link ^ (uintptr_t)new_item; //сдвигаем текущий
элемент head на вторую позицию (теперь элемент связан только с new_item)
        new_item->link = (uintptr_t)lst->head; //обновляем link new_item, чтобы он был связан с
вторым элементом списка
        lst->head = new_item; //обновляем адрес head, чтобы теперь он указывал на новый элемент
    }
    else { //случай, когда список пустой
        assert(!lst->tail);
        lst->head = lst->tail = new_item;
    }
}
}

```

//удаление одного элемента с конца

```

void pop_back(list_t *lst) {
    if (lst->head == lst->tail && lst->tail != 0) { //остался последний элемент в списке
        free(lst->tail);
        lst->tail = 0;
        lst->head = 0;
    }
    else if (lst->tail != 0 && lst->head != 0) {
        list_item_t *tmp = lst->tail; //запишем хвост в новый элемент (его нам и нужно
удалить)
        lst->tail = (list_item_t *)lst->tail->link; //новый хвост - предпоследний элемент
списка
        lst->tail->link = (uintptr_t)lst->tail->link ^ (uintptr_t)tmp; //меняем link хвоста
        free(tmp);
    }
}
}

```

//удаление одного элемента впереди

```
void pop_front(list_t *lst) {  
    if (lst->head == lst->tail && lst->head != 0) { //остался последний элемент в списке  
        free(lst->head);  
        lst->tail = 0;  
        lst->head = 0;  
    }  
    else if (lst->tail != 0 && lst->head != 0) {  
        list_item_t *tmp = lst->head; //запишем голову в новый элемент (ее нам и нужно  
удалить)  
        lst->head = (list_item_t *)lst->head->link; //новая голова - предпоследний элемент  
списка  
        lst->head->link = (uintptr_t)lst->head->link ^ (uintptr_t)tmp; //меняем link головы  
        free(tmp);  
    }  
}
```

//переделывание списка в массив для дальнейшей сортировки

```
void xor_list_to_array(list_item_t* head, char* array[], int size) {  
    list_item_t* prev = NULL; //предыдущий элемент списка  
    list_item_t* current = head; //текущий элемент списка  
    list_item_t* next; //следующий элемент списка  
    int i = 0;  
  
    while (current != NULL && i < size) {  
        array[i] = current->data;  
        next = (list_item_t*)((uintptr_t)prev ^ (uintptr_t)current->link);  
        prev = current;  
        current = next;  
        i++;  
    }  
}
```

//функции для merge sort

//функция для сравнения двух строк

```
int compare(const void *a, const void *b) {  
    const char *strA = *(const char **)a;  
    const char *strB = *(const char **)b;  
    return strcmp(strA, strB);  
}
```

//функция для слияния двух подмассивов

```
void merge(char **arr, int left, int mid, int right) {  
    int i, j, k;  
    int n1 = mid - left + 1;
```



```

int n2 = right - mid;
//создание временных подмассивов
char **L = (char **)malloc(n1 * sizeof(char *));
char **R = (char **)malloc(n2 * sizeof(char *));
// Копирование данных во временные подмассивы
for (i = 0; i < n1; i++)
    L[i] = arr[left + i];
for (j = 0; j < n2; j++)
    R[j] = arr[mid + 1 + j];
// Слияние временных подмассивов обратно в основной массив
i = 0;
j = 0;
k = left;
while (i < n1 && j < n2) {
    if (compare(&L[i], &R[j]) >= 0) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}
// Копирование оставшихся элементов L[], если они есть
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}
// Копирование оставшихся элементов R[], если они есть
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
// Освобождение памяти, выделенной для временных массивов
free(L);
free(R);
}

```

```

// Функция для сортировки слиянием
void mergeSort(char **arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

```

```

        // Рекурсивная сортировка двух подмассивов

```

```

mergeSort(arr, left, mid);
mergeSort(arr, mid + 1, right);

// Слияние отсортированных подмассивов
merge(arr, left, mid, right);
}
}

// Функция для записи хог-связного списка в бинарный файл
void write_to_file(const char* filename, list_t *lst) {
    // Открываем файл для записи, очищаем его
    FILE* file = fopen(filename, "wb");
    ftruncate(fileno(file), 0); //очистка бинарного файла

    // Переменные для хранения текущего и предыдущего узлов
    list_item_t* current = lst->head;
    list_item_t* prev = NULL;

    unsigned short index = 0;
    unsigned short size = 17;

    // Записываем данные в файл
    while (current != NULL) {
        // Записываем индекс строки (2 байта)
        //unsigned short index = (unsigned short)(current - head);
        fwrite(&index, sizeof(unsigned short), 1, file);

        // Записываем размер строки (2 байта)
        //unsigned short size = (unsigned short)current->size;
        fwrite(&size, sizeof(unsigned short), 1, file);

        // Записываем строку
        fwrite(current->data, sizeof(char), size, file);

        // Переходим к следующему узлу
        list_item_t* next = (list_item_t*)((uintptr_t)prev ^ (uintptr_t)current->link);
        prev = current;
        current = next;
        index++;
    }

    // Закрываем файл
    fclose(file);
}

int main(int argc, char *argv[]) {

```

```

char *DEBUG = getenv("LAB3DEBUG");
if (DEBUG) {
    fprintf(stderr, "Включен вывод отладочных сообщений\n");
}

//обработка входных данных
bool flag = true;
errno = 0;
if (argc == 1) {
    printf("Ошибка: не задано имя файла.\n");
    errno = ERROR_NO_FILE;
    flag = false;
}
else if (argc == 2) {
    if (strcmp(argv[1], "-v") == 0) {
        printf("Жук Анастасия Валерьевна, гр. N3148\nВариант: 3-3-9-6\n");
        flag = false;
    }
}
else if (argc > 2) {
    printf("Ошибка: много аргументов.\n");
    errno = ERROR_MANY_ARGUMENTS;
    flag = false;
}

//считывание команд
if (flag) {
    int* number = (int *) malloc(sizeof(int)); //индексы элементов в списке
    int* size = (int *) malloc(sizeof(int)); //размер входных строк
    int* place = (int *) malloc(sizeof(int)); //место нахождения строки в файле
    int i = 0, n = 0, count = 0;
    int j = 0; //количество строк в файле состояния списка

    int file_size = 0;
    FILE* bin_file = fopen(argv[1], "rb");
    if (bin_file == NULL) {
        bin_file = fopen(argv[1], "wrb");
    }
    while(getc(bin_file) != EOF) file_size++; //находим размер файла
    fseek(bin_file, 0, SEEK_SET); //fseek позволяет нам вернуться в начало файла
    //считываем индексы, размеры строк и их положение в файле, для того, чтобы в
    //дальнейшем положить эти строки в список в заданном порядке
    while (count < file_size) {
        j++;
        number = (int *) realloc(number, j * sizeof(int));
        size = (int *) realloc(size, j * sizeof(int));
    }
}

```

```

        place = (int *) realloc(place, j * sizeof(int));
        fread(&i, 2, 1, bin_file);
        fread(&n, 2, 1, bin_file);
        count += 4 + n;
        fseek(bin_file, count, SEEK_SET);
        number[j - 1] = i;
        size[j - 1] = n;
        place[j - 1] = count;
    }
    fseek(bin_file, 0, SEEK_SET);

```

1, 2, 3) //сортировка строк (selection sort) в правильном порядке (индексы: 0, 2, 3, 1 -> 0,

```

    int buf;
    for (i = 0; i < j; i++) {
        int min_index = i;
        for (int h = i + 1; h < j; h++) {
            if (number[h] < number[min_index]) min_index = h;
            if (min_index != i) { //одновременно сортируем элементы трех
                списков (элементы на i-ых позициях относятся к одной строке)
                buf = number[i];
                number[i] = number[min_index];
                number[min_index] = buf;

                buf = size[i];
                size[i] = size[min_index];
                size[min_index] = buf;

                buf = place[i];
                place[i] = place[min_index];
                place[min_index] = buf;
            }
        }
    }
}

```

```

//проверка на правильность исходных строчек в файле состояния списка
char* str = (char*)malloc(17 * sizeof(char)); //исходная строка
list_t lst = {0}; //XOR-связный список
for (i = 0; i < j; i++) {
    if (size[i] == 17) {
        fseek(bin_file, place[i] - 17, SEEK_SET);
        for (int o = 0; o < 17; o++) {
            str[o] = fgetc(bin_file);
        }
        fseek(bin_file, 0, SEEK_SET);
        bool f = true;
    }
}

```

```

        for (int h = 0; h < 17; h++) {
            if (((h + 1) % 3 != 0 && ((int) str[h] < 48 || ((int) str[h] > 57 &&
(int) str[h] < 65) || ((int) str[h] > 70 && (int) str[h] < 97) || ((int) str[h] > 102))) || ((h + 1) % 3 == 0
&& str[h] != ':')) {

                f = false;
                break;
            }
        }
        if (f) push_back(&lst, str);
        else {
            printf("Ошибка: некорректный тип данных в исходном
файле!\n");

            errno = ERROR_WRONG_TYPE_IN_FILE;
            return errno;
        }
    }
    else {
        printf("Ошибка: некорректный тип данных в исходном файле!\n");
        errno = ERROR_WRONG_TYPE_IN_FILE;
        return errno;
    }
}

```

```

//выполнение команд
int len = j; //длина списка
int t, k = 0;
char c;
int l = 0;
char *s = malloc(sizeof(char)); //строка с командой и строками, если они есть
char **arr = malloc(sizeof(char)); //массив с командой и строками, если они есть
while ((t = getchar()) != EOF) {
    c = (char) t;
    if (c != '\n') {
        l++;
        s = (char *) realloc(s, l * sizeof(char));
        s[l - 1] = c;
    }
    else {
        arr = (char **) realloc(arr, l * sizeof(char));
        char *p = strtok(s, " ");
        while (p) {
            k++;
            arr[k - 1] = p;
            p = strtok(NULL, " ");
        }
        if (strcmp(arr[0], "push_front") == 0) {

```

```

        bool f = true;
        for (i = k - 1; i > 0; i--) {
            if (strlen(arr[i]) == 17) {
                for (j = 0; j < 17; j++) {
                    if (((j + 1) % 3 != 0 && ((int) arr[i][j] <
48 || ((int) arr[i][j] > 57 && (int) arr[i][j] < 65) || ((int) arr[i][j] > 70 && (int) arr[i][j] < 97) || ((int)
arr[i][j] > 102))) || ((j + 1) % 3 == 0 && arr[i][j] != ':')) {

                        f = false;
                        break;
                    }
                }
                if (f) {
                    push_front(&lst, arr[i]);
                    len++;
                }
                else {
                    printf("Ошибка: некорректный тип
данных %s.\n", arr[i]);

                    errno = ERROR_WRONG_TYPE;
                }
            }
            else {
                printf("Ошибка: некорректный тип данных
%s.\n", arr[i]);

                errno = ERROR_WRONG_TYPE;
            }
            f = true;
        }
    }
    else if (strcmp(arr[0], "push_back") == 0) {
        bool f = true;
        for (i = 1; i < k; i++) {
            if (strlen(arr[i]) == 17) {
                for (j = 0; j < 17; j++) {
                    if (((j + 1) % 3 != 0 && ((int) arr[i][j] <
48 || ((int) arr[i][j] > 57 && (int) arr[i][j] < 65) || ((int) arr[i][j] > 70 && (int) arr[i][j] < 97) || ((int)
arr[i][j] > 102))) || ((j + 1) % 3 == 0 && arr[i][j] != ':')) {

                        f = false;
                        break;
                    }
                }
                if (f) {
                    push_back(&lst, arr[i]);
                    len++;
                }
                else {

```

```

        printf("Ошибка: некорректный тип
данных %s.\n", arr[i]);
        errno = ERROR_WRONG_TYPE;
    }
}
else {
    printf("Ошибка: некорректный тип данных
%s.\n", arr[i]);
    errno = ERROR_WRONG_TYPE;
}
f = true;
}
}
else if (strcmp(arr[0], "pop_front") == 0) {
    if (k == 1) {
        pop_front(&lst);
        if (len > 0) len--;
    }
    else {
        printf("Ошибка: лишние аргументы у команды
pop_front.\n");
        errno = ERROR_UNNECESSARY_ARGUMENTS;
    }
}
else if (strcmp(arr[0], "pop_back") == 0) {
    if (k == 1) {
        pop_back(&lst);
        if (len > 0) len--;
    }
    else {
        printf("Ошибка: лишние аргументы у команды
pop_back.\n");
        errno = ERROR_UNNECESSARY_ARGUMENTS;
    }
}
else if (strcmp(arr[0], "reorder_dsc") == 0) { //сортирует список через
массив с помощью merge sort
    if (k == 1) {
        char* array[len];
        xor_list_to_array((&lst)->head, array, len);
        mergeSort(array, 0, len - 1);
        list_t lst_new = {0};
        for (int g = 0; g < len; g++) push_back(&lst_new,
array[g]);
        lst = lst_new;
    }
}
}

```

```

else {
    printf("Ошибка: лишние аргументы у команды
reorder_dsc.\n");
    errno = ERROR_UNNECESSARY_ARGUMENTS;
}
}
else if (strcmp(arr[0], "dump") == 0) { //выводит текущее состояние
списка
    if (k == 1) {
        if (!(&lst)->head) printf("\nПустой список.");
        list_item_t *li = (&lst)->head,
            *pi = NULL,
            *tmp = NULL;
        printf("\n");
        while (li != NULL) {
            printf("%p %p %s\n", li, (list_item_t*)li->link, li-
>data);

            tmp = li;
            li = (list_item_t*)(li->link ^ (uintptr_t)pi);
            pi = tmp;
        }
        printf("\n");
    }
    else if (k == 2) {
        FILE* file = fopen(arr[1], "w");
        if (file == NULL) {
            printf("Ошибка: не удалось открыть файл для
записи.\n");

            errno = ERROR_DONT_OPEN_FILE;
        }
        if (!(&lst)->head) printf("\nПустой список.");

        list_item_t *li = (&lst)->head,
            *pi = NULL,
            *tmp = NULL;
        while (li != NULL) {
            fprintf(file, "%p %p %s\n", li, (list_item_t*)li-
>link, li->data);

            tmp = li;
            li = (list_item_t*)(li->link ^ (uintptr_t)pi);
            pi = tmp;
        }
        fclose(file);
    }
    else {

```



```

dump.\n");

printf("Ошибка: лишние аргументы у команды
errno = ERROR_UNNECESSARY_ARGUMENTS;
}

}
else {
printf("Ошибка: неподдерживаемая команда %s.\n", arr[0]);
errno = ERROR_BAD_OPERATE;
}
free(s);
s = (char *) calloc(1, 1);
l = 0;
explicit_bzero(arr, 0);
free(arr);
arr = (char**) malloc(sizeof(char));
k = 0;
}
}
fclose(bin_file);
free(s);
free(arr);
write_to_file(argv[1], &lst);
free(str);
free(number);
free(size);
free(place);
}
return errno;
}

```